

Module : Kotlin (M202)

TP N° 25

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVOAM201 - DEVOAM202 Niveau : 2ème année

Exercice 1 :

Vous allez créer une petite application Android qui simule le fonctionnement d'une voiture avec un moteur et un réservoir de carburant. La voiture dépend d'un réservoir pour connaître le niveau de carburant et d'un moteur pour démarrer.

Partie 1 : Sans Hilt (injection de dépendances manuelle)

Classes à créer :

1. **Carburant** : Représente le type de carburant utilisé par la voiture.
 - Méthode : `typeDeCarburant()` qui retourne un String représentant le type de carburant (ex: "Essence").
2. **Réservoir** : Contient le carburant et permet de vérifier son niveau.
 - Dépendance : Carburant
 - Méthode : `verifierNiveau()` qui retourne un String indiquant le type de carburant dans le réservoir.
3. **Moteur** : Démarre la voiture.
 - Méthode : `demarrer()` qui retourne un String indiquant que le moteur démarre.
4. **Voiture** : Dépend du moteur et du réservoir pour fonctionner.
 - Dépendances : Moteur, Réservoir
 - Méthode : `conduire()` qui démarre le moteur et vérifie le niveau du réservoir avant de retourner un message indiquant que la voiture est en route.

Exigences :

- Créez les objets Carburant, Réservoir, Moteur et injectez-les manuellement dans l'objet Voiture.

- Dans une activité Android (MainActivity), affichez un message dans la console indiquant que la voiture roule, en appelant la méthode conduire().

Partie 2 : Avec Hilt (injection de dépendances automatique)

Étapes à suivre :

1. **Configuration de Hilt :**
 - Ajoutez les dépendances Hilt à votre fichier build.gradle.
 - Créez une classe Application annotée avec @HiltAndroidApp.
2. **Modifications des classes :**
 - Ajoutez l'annotation @Inject au constructeur de chaque classe qui peut être injectée (Carburant, Réservoir, Moteur, Voiture).
3. **Utilisation de Hilt dans l'activité :**
 - Utilisez l'annotation @AndroidEntryPoint sur l'activité.
 - Injectez les dépendances automatiquement dans l'activité avec l'annotation @Inject sur les champs correspondants (par exemple Voiture).

Exercice 2 :

vous allez créer une application de messagerie qui gère des utilisateurs et des messages. L'application doit permettre de créer des utilisateurs, de leur attribuer des messages, et d'envoyer des messages.

Gestion des Utilisateurs dans une Application de Messagerie avec et sans Hilt

Dans cet exercice, vous allez créer une application de messagerie qui gère des utilisateurs et des messages. L'application doit permettre de créer des utilisateurs, de leur attribuer des messages, et d'envoyer des messages. L'objectif est d'implémenter cette fonctionnalité en deux

étapes : d'abord sans utiliser Hilt (injection de dépendances manuelle), puis en utilisant Hilt (injection automatique).

Partie 1 : Sans Hilt (injection de dépendances manuelle)

Classes à créer :

1. **Utilisateur** : Représente un utilisateur avec un nom.
 - Propriétés : nom (String)
 - Méthode : getNom() qui retourne le nom de l'utilisateur.
2. **Message** : Représente un message envoyé par un utilisateur.
 - Propriétés : contenu (String), expéditeur (Utilisateur)
 - Méthode : afficherMessage() qui retourne un String sous la forme : "Message de [Nom de l'utilisateur] : [Contenu]".
3. **ServiceDeMessagerie** : Permet d'envoyer un message d'un utilisateur.
 - Méthode : envoyerMessage() qui prend en paramètre un Utilisateur et un Message et affiche le message envoyé dans la console.
4. **MessagerieApplication** : Gère l'interaction entre les utilisateurs et le service de messagerie.
 - Propriétés : serviceDeMessagerie (dépendance de ServiceDeMessagerie)
 - Méthode : envoyer() qui prend en paramètre un Utilisateur et un Message et utilise le service de messagerie pour envoyer le message.

Partie 2 : Avec Hilt (injection de dépendances automatique)

Étapes à suivre :

1. **Configuration de Hilt** :
 - Ajoutez les dépendances Hilt à votre fichier build.gradle.
 - Créez une classe Application annotée avec @HiltAndroidApp.
2. **Modifications des classes** :
 - Ajoutez l'annotation @Inject au constructeur de chaque classe qui doit être injectée (ServiceDeMessagerie, MessagerieApplication).

- Utilisez l'annotation @Singleton sur ServiceDeMessagerie pour garantir qu'il existe une seule instance de ce service.